

Data Engine

Intern Project

Quant Singularity

Sai Pavan Sahu

Data Engine Track · Independent Quant Developer

Hyderabad, India

May 2026

SECTION 01

Warehouse Design and Scale

I built the warehouse as partitioned Parquet files with DuckDB views on top:

```
warehouse/  
|-- spot/date=YYYY-MM-DD/part-0.parquet  
|-- futures/date=YYYY-MM-DD/part-0.parquet  
|-- vix/date=YYYY-MM-DD/part-0.parquet  
|-- options/date=YYYY-MM-DD/expiry=YYYY-MM-DD/part-0.parquet  
`-- aux/  
    |-- fii_dii_flow.parquet  
    `-- nse_calendar.parquet
```

The strategy team usually slices by trading date, so spot, futures, and VIX are partitioned by date. That lets DuckDB prune most files for date-range queries. The feature engine has a different pattern: it asks for many timestamp-level feature rows at once and joins spot, futures, VIX, daily FII/DII flow, and options features. Date partitioning keeps those joins local to the relevant trading sessions.

Options also use an **expiry** partition because the common access pattern is a full chain for one date and one weekly or monthly expiry. A query such as *weekly chain for 2025-08-26 at 10:15* should read one date/expiry folder, not scan every chain file.

SCALE NOTE

This layout is fine for the seven-day sample, but it would be the first thing to revisit for two years of 1-minute options data across all NSE instruments. The selected date/expiry partition would become too wide across symbols, strikes, and sides. The fix is a symbol/instrument partition above date, separate weekly and monthly chain tables if query patterns justify it, and files sorted by timestamp, expiry, strike, and side. Versioned snapshot folders would also let backtests pin an exact data version.

SECTION 02

What the Validation Module Found

The full machine-readable output is in **validation_report.json**. Across the raw bundle, the module found:

21,151

Critical rows dropped

2,088

Warning rows dropped

750

Futures labels fixed

Spot data was clean.

All seven nifty_spot files passed the checks for session boundaries, zero or negative OHLC values, duplicate timestamps, and impossible candles. I investigated a sharp Aug 26 move around 10:44 because it looked suspicious, but futures moved at the same time and held the new level, so I kept the rows.

Futures — rollover label bug on Sep 01 and Sep 02.

The near_month_expiry column still showed **2025-08-28** on 2025-09-01 and 2025-09-02, even though that contract had already expired. This affected 375 rows per day, 750 rows total. I patched the label to 2025-09-25 so downstream joins do not keep treating an expired contract as the active near-month.

Futures — Aug 28 expiry day basis convergence.

On 2025-08-28, the near/mid futures correlation dropped to 0.9882. I logged this as INFO rather than treating it as a data error. The near-month contract was expiring, and the basis converged from +42.46 at 09:15 to +2.91 near close, which is plausible expiry-day behaviour.

Options — pre-open snapshots in every file.

Every options file contained three pre-open snapshots at 09:00, 09:05, and 09:10. With 29 strikes and two sides, that is 174 rows per file, or 2,088 rows total across all 12 files. I dropped them because they are auction artefacts, not continuous market prices.

Options — 21,151 corrupt OHLC candles during market hours.

After pre-open rows were removed, 21,151 impossible OHLC candles remained — bars where $high < \max(open, close)$ or $low > \min(open, close)$. I dropped them as critical because there is no defensible way to know which field is wrong.

Options — complete IV feed failure on Aug 28 expiry day.

CRITICAL FINDING

The 2025-08-28 weekly expiry options file had IV frozen at 0.1540 across 2,604 retained price/OI rows after pre-open and corrupt candles were handled — every strike, both CE and PE, from 09:15 to 15:25. I set IV to null and marked `iv_reliable=False` for those rows, while keeping price and OI because the IV failure does not automatically invalidate the traded fields.

Summary of all findings by severity:

Severity	Finding	Decision
INFO	Spot Aug 26 10:44 spike +1.59% — confirmed real by futures cross-reference	Kept
INFO	Aug 28 near/mid correlation 0.9882 — expiry day basis convergence	Kept, logged
FIXED	Sep 01 & Sep 02 near_month_expiry='2025-08-28' post-expiry label	Patched to 2025-09-25 (750 rows)
WARNING	Pre-open snapshots 09:00–09:10 in all 12 options files	Dropped (2,088 rows)
CRITICAL	Corrupt OHLC candles during market hours across all options files	Dropped (21,151 rows)
CRITICAL	IV frozen at 0.1540 across 2,604 rows in Aug 28 weekly expiry file	IV nulled, iv_reliable=False

SECTION 03

Access Function Contracts

`get_price(timestamp, instrument)`

Returns the close for the requested 1-minute bar. Accepts **spot**, **near_futures**, or **mid_futures**. Before 09:15 returns None; after 15:29 caps to the 15:29 close. Seconds are floored to the minute. Missing bars return None rather than being forward-filled.

`get_features(timestamp, lookback_days=30)`

Returns a dictionary with spot close, session VWAP, VIX, FII/DII flow, near futures basis, ATM CE/PE IV, PCR OI, **lookback_complete**, and **partial**. In the first five minutes of the session, VWAP uses the bars available so far. Because the sample has only seven trading days, the default 30-day lookback sets lookback_complete=False, but the function still returns usable features. If one source is missing, partial=True.

`get_signals(chain_type, timestamp)`

Returns the weekly or monthly options chain snapshot at or before the requested time. If the timestamp falls between two five-minute snapshots, it returns the previous snapshot and marks **stale=True**. It does not interpolate options prices or IV. Pre-open timestamps return None.

`get_features_batch(timestamps, lookback_days=30)`

Returns one row per input timestamp in the same order, including duplicates. Invalid timestamps are returned with **valid=False** and an `invalid_reason`. Missing fields become null and set `partial=True`. The batch function fills the same feature columns as `get_features`, including ATM CE/PE IV where the IV feed is reliable.

SECTION 04

Benchmark Results

Benchmarks were run on Windows 10, Intel64 Family 6 Model 142, with DuckDB reading local Parquet files. The single-access functions used 100 distinct timestamps, and the batch function used 1,000 timestamps.

Function	Samples	Median	p99	95% CI (mean)
<code>get_price("spot")</code>	100	4.41 ms	12.86 ms	4.37 – 5.65 ms
<code>get_features</code>	100	57.86 ms	70.30 ms	58.13 – 60.39 ms
<code>get_signals("weekly")</code>	100	23.11 ms	32.20 ms	22.82 – 25.08 ms
<code>get_features_batch(1000)</code>	1,000	—	—	26.03 s wall time

The relative ordering makes sense. **get_price** is a narrow single-row lookup. **get_signals** reads a full chain snapshot. **get_features** is slower because it touches multiple sources and computes options-derived fields. The p99 values are mainly from heavier options lookups and cold file reads.

KNOWN LIMITATION

The batch result is honest but not production-ready. The function is contract-correct, but it still loops for VWAP and options-derived features. Replacing those loops with grouped SQL window functions and one options aggregation plan per date is the next step before calling this production-grade.

SECTION 05

Results

The raw bundle has 29 CSV files: 7 spot, 7 futures, 12 options-chain files, and 3 auxiliary files. The warehouse rebuilds into deterministic Parquet paths with fixed filenames, which makes rerun comparisons easier.

Source	Files	Result
Spot	7	Clean; all rows kept
Futures	7	Kept; 750 stale near-expiry labels patched
Options	12	Pre-open and corrupt OHLC rows dropped; frozen IV marked unreliable
VIX	1	Clean after validation
FII/DII and calendar	2	Converted to Parquet

The latest benchmark output is in **benchmark.csv** and the validation output is in **validation_report.json**. The main gap is not that the layer fails — it is that the current batch implementation is still too slow for a feature engine that needs thousands of timestamps repeatedly.

SECTION 06

How I Know This Layer Is Correct

The strongest evidence is not that the functions return values. It is that the layer refuses to silently accept bad boundaries: pre-open rows are dropped, impossible OHLC rows are removed, frozen IV is marked unreliable, and missing values are exposed through None, nulls, partial, or valid=False.

What the validation module cannot catch.

- CE and PE labels being swapped if the numbers remained plausible.
- A futures price that is wrong by 50 points but still inside a broad daily range.
- A missing strike unless expected-chain completeness rules are added.

- Put-call parity violations, IV smile continuity issues, or basis bounds relative to days-to-expiry.

Adversarial timestamps tested.

A pre-open timestamp returns None for single-access functions and an invalid row for the batch API. On futures expiry day, the basis anomaly is logged rather than hidden. If an options timestamp falls between snapshots, the previous snapshot is returned and marked stale. If IV is unreliable, ATM IV stays null instead of pretending the vendor value is usable.

Why the benchmark numbers are optimistic.

Seven trading days are small, and repeated local runs benefit from OS cache. With six months of options data, partition counts grow, metadata reads get heavier, and p99 latency would likely rise. The current batch function would break first because it still does too much work per timestamp.

What I would build next.

- Row and file checksums on Parquet files to detect silent on-disk corruption.
- Snapshot-versioned warehouse paths so backtests can pin an exact data version.
- Put-call parity checks, IV smile continuity checks, and missing-strike completeness checks.
- A live freshness monitor that alerts if the latest timestamp is stale by more than 2 minutes.
- Grouped SQL window functions to replace the VWAP and options loops in `get_features_batch`.

PRODUCTION RISK

In a live trading environment, the most likely failure is feed drift: changed columns, changed timestamp conventions, missing chain files, or a partial trading day that looks valid until a strategy consumes it. The next version should fail loudly on those conditions before any downstream system can consume plausible-looking but wrong data.